Операционные системы

Лекция 4

Управление процессами

План лекции

- Понятие «Процесс»
- Ресурсы доступные процессу
- Создание и завершение процессов
- Функции получения информации о процессе
- Межпроцессное взаимодействие

Процесс

Процессы – это одна из самых старых и наиболее важных абстракций, присущих операционной системе

Они поддерживают возможность осуществления (псевдо) параллельных операций даже при наличии всего одного центрального процессора. Они превращают один центральный процессор в несколько виртуальных

Без абстракции процессов современные вычисления просто не могут существовать

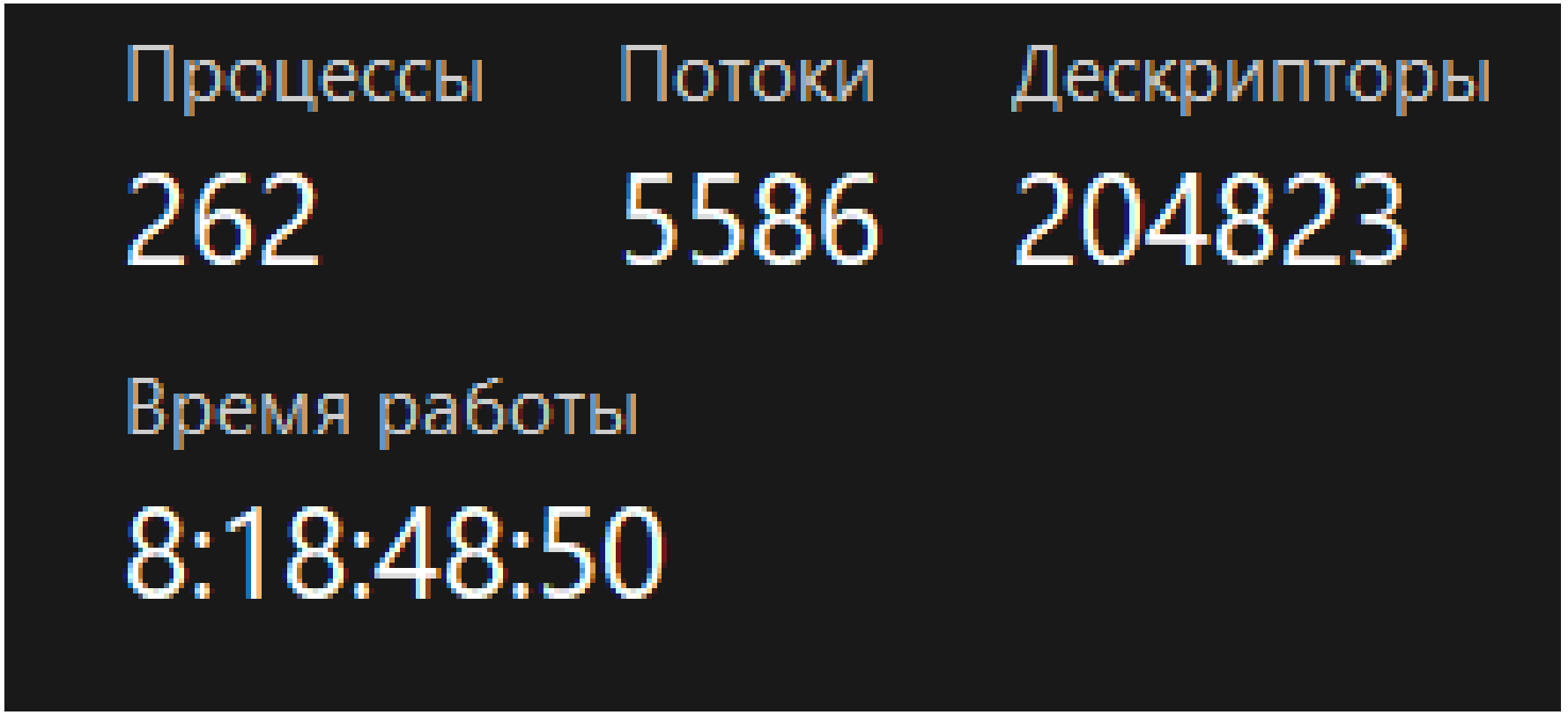
Поэтому стоит начать наше знакомство с миром абстракций предоставляемых ОС именно с них, так что вперёд!

Процесс

В любой многозадачной системе центральный процессор быстро переключается между процессами, предоставляя каждому из них десятки или сотни миллисекунд

При этом хотя в каждый конкретный момент времени центральный процессор работает только с одним процессом, в течение 1 секунды он может успеть поработать с несколькими из них, создавая иллюзию параллельной работы

Иногда в этом случае говорят о **псевдопараллелизме** в отличие от настоящего аппаратного параллелизма в многопроцессорных системах (у которых имеется не менее двух центральных процессоров, использующих одну и ту же физическую память)


Процессы
Потоки
Дескрипторы
262
5586
204823
Время работы
8:18:48:50

Процесс

Людям довольно трудно отслеживать несколько действий, происходящих параллельно. Поэтому разработчики ОС создали **концептуальную модель последовательных процессов**

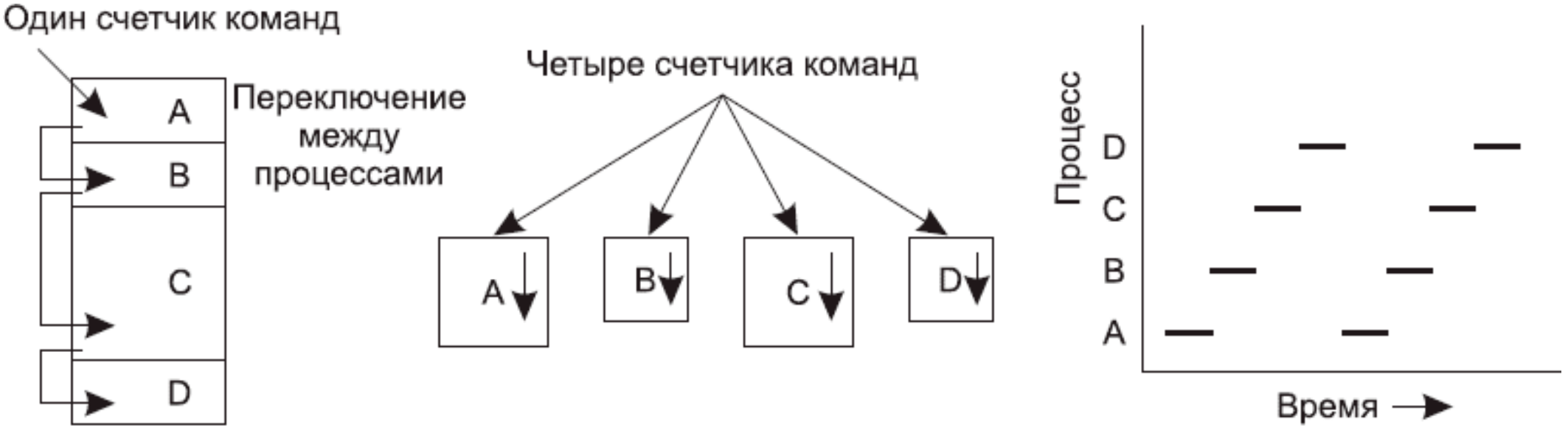
В этой модели все выполняемое на компьютере программное обеспечение, иногда включая операционную систему, сведено к ряду последовательных процессов, или, для краткости, просто процессов

Если говорить по простому, то процесс – это просто экземпляр выполняемой программы, включая текущие значения счётчика команд, регистров и переменных

Процесс

Как видно из представленных схем, процессор постоянно переключается между процессами (хотя каждый процесс считает, что весь процессор принадлежит только ему)

Такое постоянное переключение между процессами называется **мультипрограммированием** или **многозадачностью**



Процесс

Подводя итоги, что же всё таки такое процесс выделим следующие два определения:

Процесс – управляющий объект, который обеспечивает изоляцию адресных пространств и представляет работающий экземпляр программы

Процесс – исполняемое на устройстве приложение весте со всеми ресурсами, которые требуются для его исполнения

Запомним! Программа и процесс не являются синонимами!

Также отметим, что довольно часто встречающееся выражение «процесс выполняется» неточно. Процессы не выполняются – они **управляют**

Процесс

Зная вашу «любовь» к классификациям, хотелось бы привести здесь что-то интересное и обширное, но к сожалению процессы разве что разделяются на **системные** и **пользовательские**

Системные процессы практически ничем не отличаются от любых написанных пользователем программ. Единственный критерий, по которому отличают прикладные процессы и системные сервисные процессы, состоит в том, что процесс операционной системы управляет каким-либо ресурсом в интересах прикладного процесса

Процесс

Процессы, которые выполняют системный код, называются ***системными*** *и применяются к системе в целом.* Они занимаются выполнением таких служебных задач, как распределение памяти, обмен страницами между внутренним и вспомогательным запоминающими устройствами, контроль устройств и т.п. Они также выполняют некоторые задачи «по поручению» пользовательских процессов, например, делают запросы на ввод-вывод данных, выделяют память и т.д.

Фоновые процессы, предназначенные для обработки какой-либо активной деятельности, называются **демонами** в Unix и **службами** в Windows

Почему именно процесс?

Но почему всё таки процессы?

Потому что они обеспечивают нам следующее:

- Инструмент обеспечения многозадачности и параллелизма
  - Процессная многозадачность
  - Поточная многозадачность
- Защита ресурсов необходимых отдельному приложению
- Модульность и абстракция
- Управление процессом обеспечивает ОС
- Обеспечение стабильности и надёжности работы системы

Процесс или поток?

Аргументы в пользу использования процессной многозадачности:

- Задачи являются независимыми
- Безопасность и изоляция отдельных задач
- Поддержка языками и библиотеками
- Ресурсоёмкие задачи
- Переносимость и масштабирование
- Устойчивость к ошибкам
- Задачи, связанные с вводом/выводом и сетью
- Длительные и сложные задачи

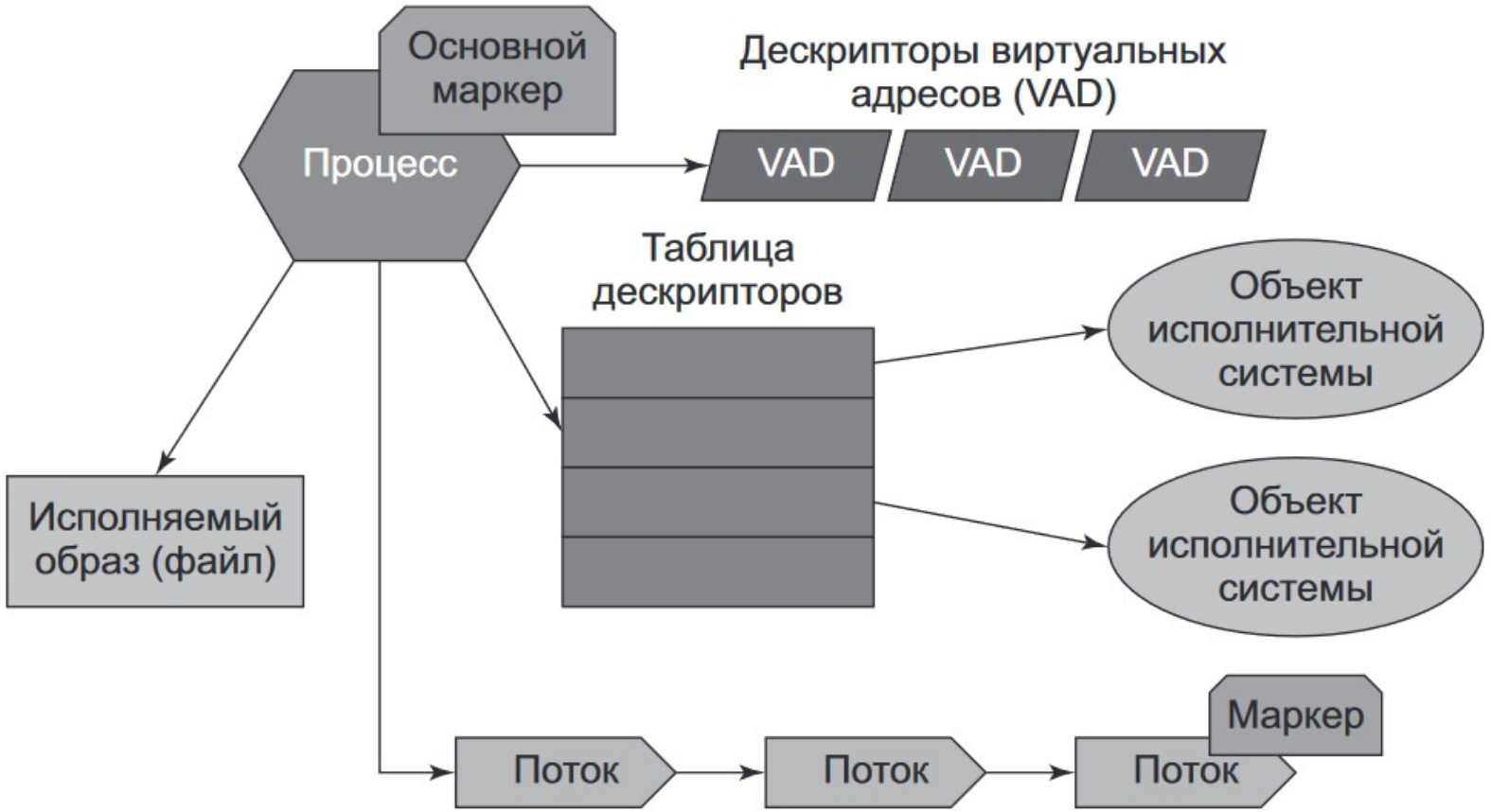
Ресурсы процесса

Теперь хотелось бы узнать, что же это всё таки за ресурсы, которые ему требуются

К типичным ресурсам процесса относятся:

- Образ исполняемого файла
- Память
- Список дескрипторов объектов выделенных процессу (файлы, потоки, объекты синхронизации и т.д.)
- Атрибуты безопасности (маркеры)
- Контекст процесса

Ресурсы процесса



Ресурсы процесса

Каждому процессу соответствует своя структура данных в ядре ОС которая представляет его и содержит важную информацию о состоянии, атрибутах и ресурсах процесса

Кроме ранее названных ресурсов в такой структуре также содержатся:

- Идентификатор процесса (PID)
- Идентификатор родительского процесса (PPID)
- Состояние процесса
- Флаги процесса
- Информация связанная с диспетчеризацией
- И т. д.

Более подробно о ней в Windows (**EPROCESS**) и Linux (**task_struct**)

Ресурсы процесса

По сути данные объекты ядра ОС включают в себя или являются реализацией структуры данных называемой **блоком управления процессом** (**PCB, Process Control Block**)

Такой блок содержит информацию о процессе. При чём любая операция, производимая операционной системой над процессом, вызывает изменения в PCB

Данные в PCB между операциями остаются постоянными

В Windows структура PCB является частью структуры EPROCESS, а в Linux – task_stuct является реализацией PCB

Ресурсы процесса

```
lkd> dt nt!_EPROCESS
   +0x000 Pcb              : _KPROCESS
   +0x438 ProcessLock      : _EX_PUSH_LOCK
   +0x440 UniqueProcessId  : Ptr64 Void
   +0x448 ActiveProcessLinks : _LIST_ENTRY
   +0x458 RundownProtect   : _EX_RUNDOWN_REF
   +0x460 Flags2           : Uint4B
   +0x460 JobNotReallyActive : Pos 0, 1 Bit
   +0x460 AccountingFolded : Pos 1, 1 Bit
   +0x460 NewProcessReported : Pos 2, 1 Bit
   +0x460 ExitProcessReported : Pos 3, 1 Bit
   +0x460 ReportCommitChanges : Pos 4, 1 Bit
   +0x460 LastReportMemory : Pos 5, 1 Bit
   +0x460 ForceWakeCharge  : Pos 6, 1 Bit
   +0x460 CrossSessionCreate : Pos 7, 1 Bit
   +0x460 NeedsHandleRundown : Pos 8, 1 Bit
   +0x460 RefTraceEnabled  : Pos 9, 1 Bit
   +0x460 PicoCreated      : Pos 10, 1 Bit
   +0x460 EmptyJobEvaluated : Pos 11, 1 Bit
   +0x460 DefaultPagePriority : Pos 12, 3 Bits
   +0x460 PrimaryTokenFrozen : Pos 15, 1 Bit
```

```
struct task_struct {
        struct thread_info      thread_info;    /*     0    24 */
        unsigned int            __state;        /*    24     4 */

        /* XXX 4 bytes hole, try to pack */

        void *                  stack;          /*    32     8 */
        refcount_t              usage;          /*    40     4 */
        unsigned int            flags;          /*    44     4 */
        unsigned int            ptrace;         /*    48     4 */
        int                     on_cpu;         /*    52     4 */
        struct __call_single_node  wake_entry;  /*    56    16 */
        /* --- cacheline 1 boundary (64 bytes) was 8 bytes ago --- */
        unsigned int            wakee_flips;    /*    72     4 */

        /* XXX 4 bytes hole, try to pack */

        long unsigned int       wakee_flip_decay_ts;  /*    80     8 */
        struct task_struct *    last_wakee;     /*    88     8 */
        int                     recent_used_cpu;  /*    96     4 */
        int                     wake_cpu;       /*   100     4 */
        int                     on_rq;          /*   104     4 */
        int                     prio;           /*   108     4 */
        int                     static_prio;    /*   112     4 */
        int                     normal_prio;    /*   116     4 */
        unsigned int            rt_priority;    /*   120     4 */

        /* XXX 4 bytes hole, try to pack */
```

Ресурсы процесса

Информация, хранимая в блоке управления процессом, делится на две части: **регистровый контекст** и **системный контекст**. **Регистровым контекстом** процесса называется содержимое всех регистров общего назначения процессора (включая значение программного счетчика)

Содержимое остальных регистров процессора называется **системным контекстом** процесса. Информации, получаемой с регистровых и системных контекстов, достаточно для управления работой процесса в операционной системе

С точки зрения пользователя, наибольший интерес вызывает вычислительная деятельность процесса, последовательность преобразования данных и полученные результаты. **Пользовательским контекстом** называются данные, находящиеся в адресном пространстве процесса

Ресурсы процесса

Совокупность регистрового, системного и пользовательского контекстов процесса называется **контекстом процесса**

Контекст процесса – минимальный набор данных, используемый процессом, который должен быть сохранен, чтобы выполнение процесса могло быть прервано и в последующем возобновлено с той же точки

Контекст процесса состоит из:

- Program Counter (PC) или Instruction Pointer (IP)
- Значений регистров
- Стека
- Кучи
- Глобальных переменных
- И т. д.

Управление процессом

Рассмотрев какую информацию включает в себя процесс, следующим вопросом является откуда берутся процессы?

Существуют четыре основных события, приводящих к созданию процессов:

- Инициализация системы
- Выполнение работающим процессом системного вызова, предназначенного для создания процесса
- Запрос пользователя на создание нового процесса
- Инициация пакетного задания

Управление процессом

За создание процесса отвечает ядро ОС

Чтобы запросить у ядра создание нового процесса необходимо совершить системный вызов

Системный вызов Windows – **NtCreateUserProcess**

Системные вызовы Linux (POSIX) – **fork**, **vfork**

Дочерний процесс – процесс создаваемый в результате системного вызова

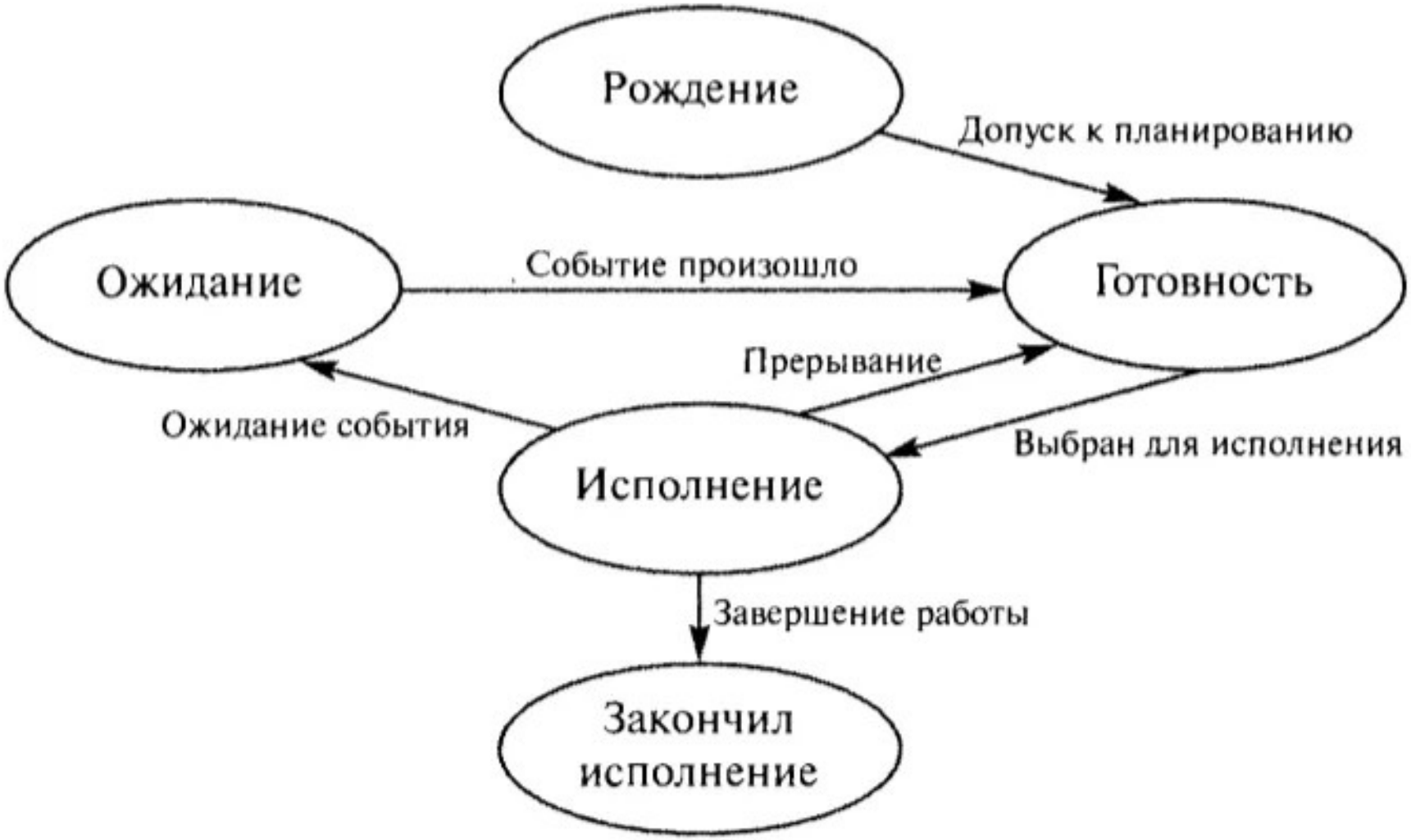
Родительский процесс – процесс инициировавший системный вызов

Управление процессом

Основными действиями ОС над процессами являются:

- **Запуск процесса**. Из числа процессов, находящихся в состоянии готовность, операционная система выбирает один процесс для последующего исполнения
- **Приостановка процесса**. Работа процесса, находящегося в состоянии исполнение, приостанавливается в результате какого-либо прерывания
- **Блокирование процесса**. Процесс блокируется, когда он не может продолжать работу, не дождавшись возникновения какого-либо события в вычислительной системе
- **Разблокирование процесса**. После возникновения в системе какого-либо события операционной системе нужно точно определить, какое именно событие произошло
- **Переключение контекста**

Управление процессом



Управление процессом

На приведенной схеме отражена обобщенная модель с созданием и завершением процесса

Состояние **рождения** – это первый этап жизненного цикла процесса. В этом состоянии происходит полное формирование процесса как системной сущности

В это время система создаёт все необходимые структуры данных, выделяет первичные ресурсы и подготавливает процесс к дальнейшей работе. Важно отметить, что до завершения этого состояния процесс ещё не готов к выполнению и не может быть запланирован на процессор

Управление процессом

В состоянии **готовности** процесс полностью подготовлен к выполнению, но ожидает своей очереди на процессор. Он находится в специальной очереди планировщика и ждёт момента, когда освободится CPU

Например, после создания нового процесса или завершения операции ввода-вывода процесс переходит в это состояние

Состояние **ожидания** возникает, когда процесс не может продолжить выполнение из-за внешних факторов

Например, при чтении файла с диска процесс переходит в это состояние, пока операция ввода-вывода не завершится. Также в это состояние процесс может перейти при ожидании сигнала от другого процесса

Управление процессом

В состоянии **исполнения** процесс получает возможность использовать процессорное время. В этот период один или несколько потоков процесса выполняют свои задачи, взаимодействуют с системными ресурсами и осуществляют обработку данных

Важно понимать, что само состояние относится к процессу как к сущности, управляющей ресурсами, в то время как фактическое выполнение кода происходит на уровне потоков

В состоянии **завершения** процесс прекращает своё существование. Все выделенные ресурсы освобождаются, а информация о завершении передаётся родительскому процессу

Например, процесс может перейти в это состояние после выполнения команды `exit()` *или при получении сигнала об аварийном завершении*

Создание процесса – Windows

Функции **CreateProcessA** и **CreateProcessW**

```
BOOL CreateProcessA(
  [in, optional]      LPCSTR                lpApplicationName,
  [in, out, optional] LPSTR                 lpCommandLine,
  [in, optional]      LPSECURITY_ATTRIBUTES lpProcessAttributes,
  [in, optional]      LPSECURITY_ATTRIBUTES lpThreadAttributes,
  [in]                BOOL                  bInheritHandles,
  [in]                DWORD                 dwCreationFlags,
  [in, optional]      LPVOID                lpEnvironment,
  [in, optional]      LPCSTR                lpCurrentDirectory,
  [in]                LPSTARTUPINFOA        lpStartupInfo,
  [out]               LPPROCESS_INFORMATION lpProcessInformation
);
```

Создание процесса – Windows

Функции **CreateProcessA** и **CreateProcessW**

```
typedef struct _STARTUPINFOA {
  DWORD  cb;
  LPSTR  lpReserved;
  LPSTR  lpDesktop;
  LPSTR  lpTitle;
  DWORD  dwX;
  DWORD  dwY;
  DWORD  dwXSize;
  DWORD  dwYSize;
  DWORD  dwXCountChars;
  DWORD  dwYCountChars;
  DWORD  dwFillAttribute;
  DWORD  dwFlags;
  WORD   wShowWindow;
  WORD   cbReserved2;
  LPBYTE lpReserved2;
  HANDLE hStdInput;
  HANDLE hStdOutput;
  HANDLE hStdError;
} STARTUPINFOA, *LPSTARTUPINFOA;
```

```
typedef struct _PROCESS_INFORMATION {
  HANDLE hProcess;
  HANDLE hThread;
  DWORD  dwProcessId;
  DWORD  dwThreadId;
} PROCESS_INFORMATION, *PPROCESS_INFORMATION,
```

Создание процесса – Windows

```
char* cmd_1 = strdup(".\\Lab-03-02x.exe 50");
STARTUPINFO si;
PROCESS_INFORMATION pi;

ZeroMemory(&si, sizeof(si));
si.cb = sizeof(si);
ZeroMemory(&pi, sizeof(pi));

if (!CreateProcessA(NULL,
                    cmd_1,
                    NULL,
                    NULL,
                    FALSE,
                    CREATE_NEW_CONSOLE | CREATE_SUSPENDED,
                    NULL,
                    NULL,
                    &si,
                    &pi)) {
  printf("CreateProcess failed (%d).\n", GetLastError());
  return 1;
}
```

Создание процесса – Windows

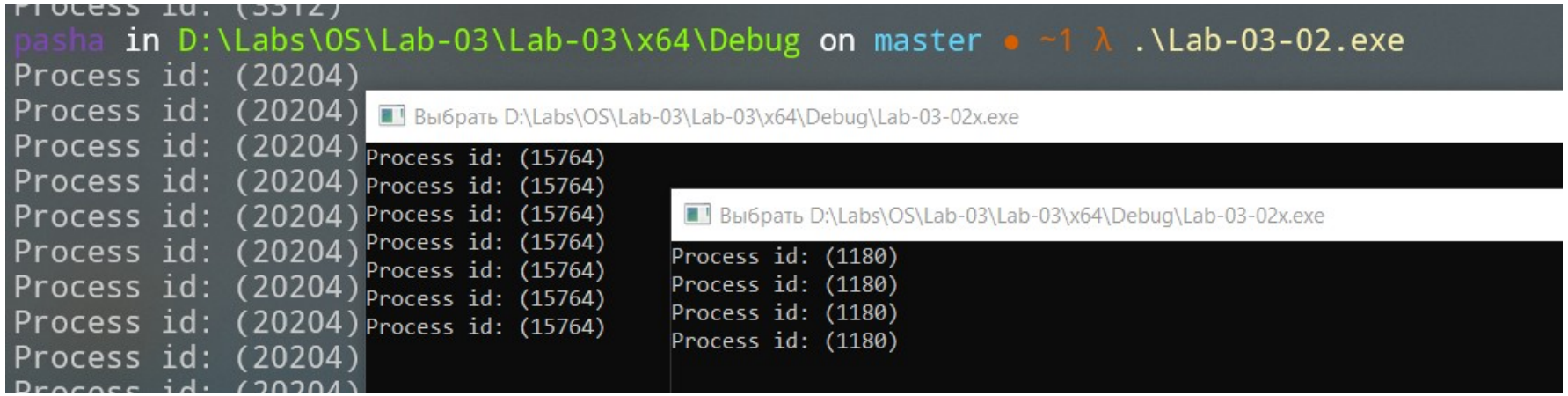
Стоит учесть что поиск указанного файла/команды осуществляется в следующем порядке:

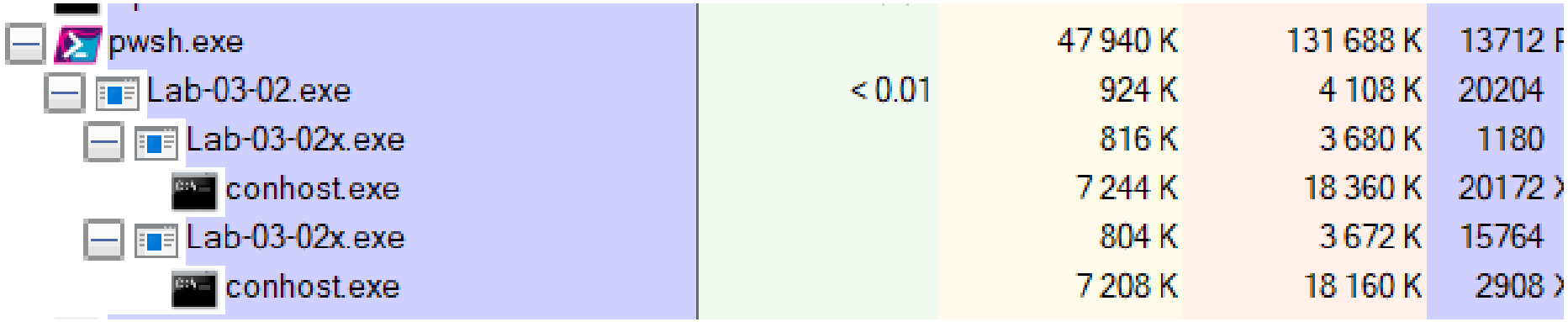
- Каталог, содержащий EXE-файл вызывающего процесса.
- Текущий каталог вызывающего процесса
- Системный каталог Windows (**GetSystemDirectory**)
- Основной каталог Windows (**GetWindowsDirectory**)
- Каталоги, перечисленные в переменной окружения PATH

Конечно, если в имени файла указан полный путь доступа, система сразу обращается туда и не просматривает эти каталоги

ПРИМЕЧАНИЕ! Данные каталоги используются системой при использовании 2-ого параметра функции CreateProcess

Создание процесса – Windows





Завершение процесса – Windows

Перед завершением процесса может возникнуть ситуация когда требуется дождаться завершения другого (напр. дочернего) процесса. В таком случае могут помочь функции **WaitForSingleObject** и **WaitForMultipleObjects**

Для бессрочного ожидания сигнала от другого процесса следует в последний параметр данных функций передать значение **INFINITE**, **А НЕ INFINITY**

```
DWORD WaitForSingleObject(
  [in] HANDLE hHandle,
  [in] DWORD  dwMilliseconds
);
```

```
DWORD WaitForMultipleObjects(
  [in] DWORD        nCount,
  [in] const HANDLE *lpHandles,
  [in] BOOL         bWaitAll,
  [in] DWORD        dwMilliseconds
);
```

Завершение процесса – Windows

Перед завершением процесса может возникнуть ситуация когда требуется дождаться завершения другого (напр. дочернего) процесса. В таком случае могут помочь функции **WaitForSingleObject** и **WaitForMultipleObjects**

```
WaitForMultipleObjects(
  2, new HANDLE[]{ pi.hThread, pi2.hThread }, TRUE, INFINITE);
```

```
WaitForSingleObject(pi.hThread, INFINITE);
WaitForSingleObject(pi2.hThread, INFINITE);
```

Завершение процесса – Windows

Функции **ExitProcess**, **TerminateProcess** и **CloseHandle**

```
void ExitProcess(
  [in] UINT uExitCode
);
```

```
BOOL TerminateProcess(
  [in] HANDLE hProcess,
  [in] UINT   uExitCode
);
```

```
BOOL CloseHandle(
  [in] HANDLE hObject
);
```

Завершение процесса – Windows

ExitProcess:

- Является функцией для **штатного** завершения работы процесса
- Происходит выгрузка всех ресурсов используемых процессом

TerminateProcess:

- Является функцией для **аварийного** завершения работы процесса
- Освобождение ресурсов процесса **не гарантируется**
- Рекомендуется использовать для завершения **зависших** процессов

Завершение процесса – Windows

CloseHandle:

- Функция делает указанный дескриптор не действительным
- Уменьшает значение счётчика дескрипторов указанного объекта
- Производит проверку на то, требуется ли освобождение объекта из системы
- Использование данной функции на дескрипторе процесса не приводит к завершению его работы
- В случае вызова функции **TerminateProcess**, требуется вручную вызвать **CloseHandle** для дескриптора завершённого процесса
- **ExitProcess** вызывает данную функцию самостоятельно

Создание процесса – Linux (POSIX)

Функции **fork** и **exec**

```
#include <unistd.h>

pid_t fork(void);
```

```
#include <unistd.h>

extern char **environ;
int execl(const char *path, const char *arg0, ... /*, (char *)0 */);
int execle(const char *path, const char *arg0, ... /*,
    (char *)0, char *const envp[]*/);
int execlp(const char *file, const char *arg0, ... /*, (char *)0 */);
int execv(const char *path, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[]);
int execvp(const char *file, char *const argv[]);
int fexecve(int fd, char *const argv[], char *const envp[]);
```

Создание процесса – Linux (POSIX)

Функция **fork** – предназначена для создания нового дочернего процесса, который будет являться полной (насколько это возможно) копией родительского процесса

Функция **exec** – предназначена для замены образа исполняемого файла в рамках существующего процесса

Важно! **Никаких новых процессов** при вызове **exec** не создаётся. Загружается **новая программа**!

По сути сбрасывается контекст процесса и загружается новый

Создание процесса – Linux

fork.c

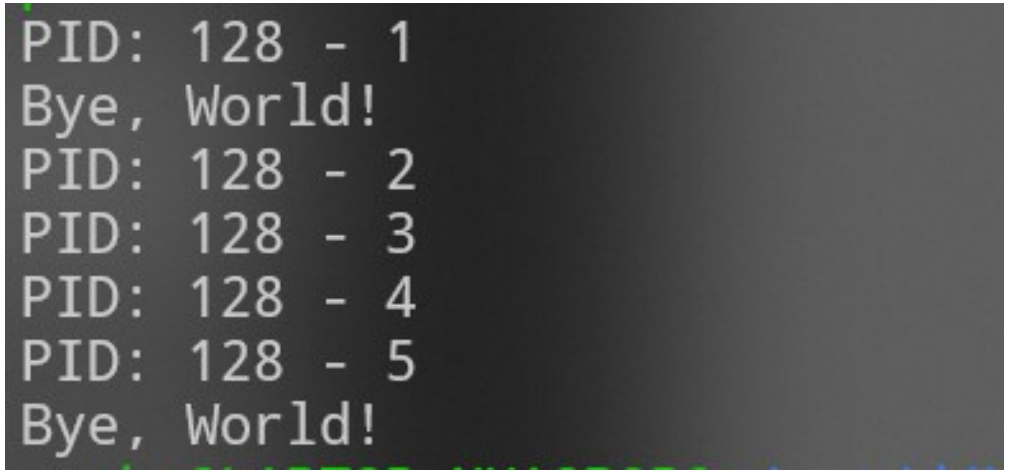
```
int
main()
{
  switch (fork()) {
    case 0:
      execl(".\next", "", (char*)NULL);
      break;
    case -1:
      printf("Fork failed!\n");
      break;
    default: {
      __pid_t pid = getpid();
      for (size_t i = 0; i < 5; i++) {
        printf("PID: %d - %zu\n", pid, i + 1);
        sleep(1);
      }
    } break;
  }
  printf("Bye, World!\n");
  return 0;
```

next.c

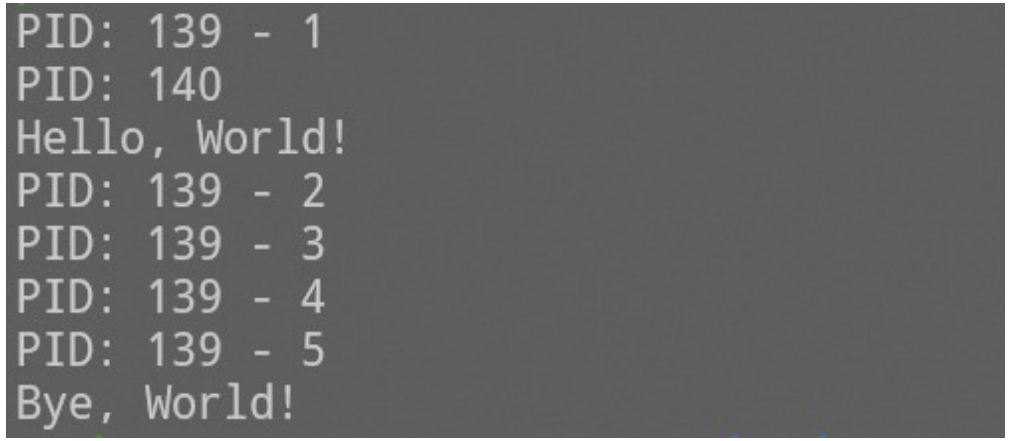
```
int
main()
{
  printf("PID: %d\n", getpid());
  printf("Hello, World!\n");
  return 0;
```

Создание процесса – Linux (POSIX)

Ошибка выполнения Exec



Успешное выполнение Exec



Создание процесса – Linux (POSIX)

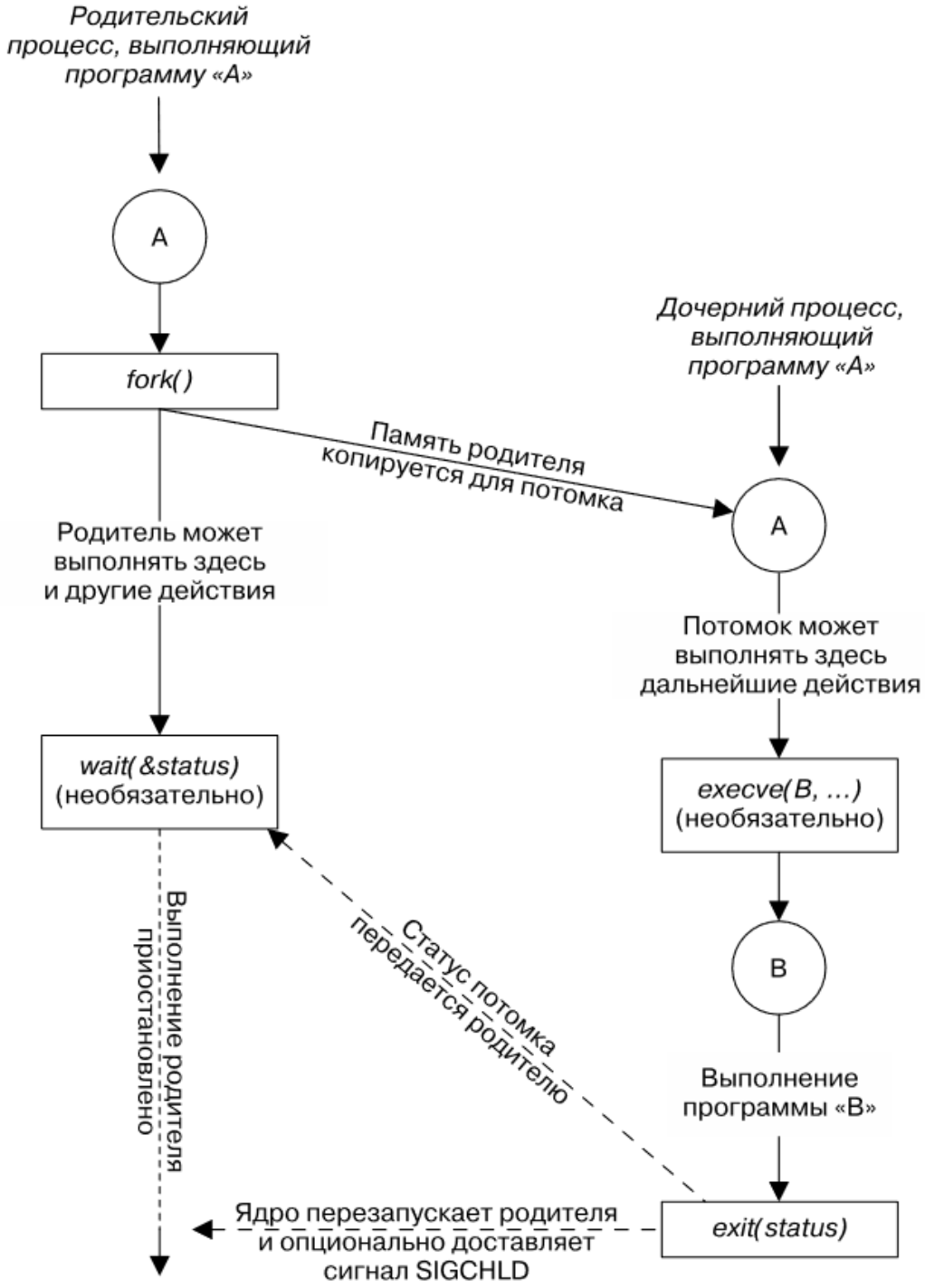
В случае с exec поиск образа происходит так:

- Если параметр path содержит символ «\», то система интерпретирует такое значение как относительный либо абсолютный путь к файлу
- В остальных случаях система проверяет каталоги, перечисленные в переменной окружения PATH на наличие указанной команды

Просмотреть информацию о всех процессах можно с помощью утилит **ps** и **top** или изучив файлы расположенные в папке соответствующего процесса (/proc/{pid})

Завершение процесса – Linux (POSIX)

- Функция **wait** – приостанавливает выполнение родителя, пока не будет завершен один из его потомков
- Библиотечная функция **exit** завершает процесс, делая все его ресурсы (память, дескрипторы открытых файлов и т. д.) доступными для последующего перераспределения ядром.



Создание процесса

Функция **system (Win/POSIX)** – позволяет вызывающей программе выполнять произвольные консольные команды

```
int system(
   const char *command
);
int _wsystem(
   const wchar_t *command
);
```

```
#include <stdlib.h>

int system(const char *command);
```

Создание процесса

Принципиальные преимущества функции **system** – это простота и удобство, а именно:

- Нам не нужно иметь дело с деталями вызовов **fork**, **exec**, **wait** и **exit** в Linux или **CreateProcess** и **WaitForSingleObject** в Windows.
- Обработка ошибок и сигналов выполняется за нас самой функцией **system**
- Поскольку **system** использует для выполнения команды командную оболочку, перед ее запуском выполняются все стандартные процедуры обработки, подстановки и перенаправления

Свойства процесса

- Процессу соответствует исполняемый программный файл
- У процесса есть PID
- У процесса есть Parent PID
- Идентификатор объекта процесса: в Windows - HANDLE и в Linux - pid_t
- В OS есть процесс инициализации (родитель для всех)
- Запуск и управление (создать, остановить,…) процессом осуществляется с помощью системных вызовов
- Процессы изолированы друг от друга

Свойства процесса

- Процессу выделяется линейное адресное пространство (размер зависит от разрядности), сегменты: code, static, data, heap, stack
- Контекст процесса – данные, которые сохраняются при переключении процессов и предназначенные для продолжения работы
- Процессу автоматически доступны три потока данных: ввода (0), вывода (1), вывода ошибок (2)
- В составе OS есть таблица, содержащая объекты ядра процессов (состояние, приоритет, указатели на другие объекты); есть средства ОС позволяющие ее просматривать
- Процесс – единица работы ОС

Стандартные потоки ввода/вывода

Стандартные потоки ввода-вывода в системах типа UNIX (и некоторых других) – потоки процесса, имеющие номер (дескриптор), зарезервированный для выполнения некоторых «стандартных» функций. Как правило (хотя и не обязательно), эти дескрипторы открыты уже в момент запуска задачи (исполняемого файла)

Как уже было сказано процессу автоматически доступны три потока данных:

- Ввода (0)
- Вывода (1)
- Вывода ошибок (2)

Стандартные потоки ввода/вывода

Поток номер **0** (**stdin**) зарезервирован для чтения команд пользователя или входных данных

При интерактивном запуске программы по умолчанию нацелен на чтение со стандартного устройства ввода (клавиатуры). Командная оболочка UNIX (и оболочки других систем) позволяют изменять цель этого потока с помощью символа «<». Системные программы (демоны и т. п.), как правило, не пользуются этим потоком

Стандартные потоки ввода/вывода

Поток номер **1** (**stdout**) зарезервирован для вывода данных, как правило (хотя и не обязательно) текстовых

При интерактивном запуске программы по умолчанию нацелен на запись на устройство отображения (монитор). Командная оболочка UNIX (и оболочки других систем) позволяют перенаправить этот поток с помощью символа «>». Средства для выполнения программ в фоновом режиме (например, nohup) обычно переназначают этот поток в файл

Стандартные потоки ввода/вывода

Поток номер **2** (**stderr**) зарезервирован для вывода диагностических и отладочных сообщений в текстовом виде.

Чаще всего цель этого потока совпадает с stdout, однако, в отличие от него, цель потока stderr не меняется при «>» и создании конвейеров («|»). То есть, отладочные сообщения процесса, вывод которого перенаправлен, всё равно попадут пользователю. Командная оболочка UNIX позволяет изменять цель этого потока с помощью конструкции «2>». Например, для подавления вывода этого потока нередко пишется 2>/dev/null

Информация о процессе

Windows	Linux (POSIX)
Получение PID и PPID	
GetCurrentProcessId **GetProcessId**	**getpid**, **getppid**
Переменные окружения	
GetEnvironmentVariable **SetEnvironmentVariable**	**getenv**, **putenv** **setenv**, **unsetenv**
Текущие диск и каталог	
GetCurrentDirectory **SetCurrentDirectory**	**getcwd** **chdir**

Справочник по WinAPI функциям для процессов: **processthreadsapi.h**

Межпроцессное взаимодействие

IPC (Inter-process communication, рус., **межпроцессное взаимодействие**) – механизм, позволяющий процессам обмениваться данными и синхронизировать свои действия

Это фундаментальная концепция системного программирования, поскольку она позволяет процессам работать вместе для достижения общей цели

Выделяются три уровня средств IPC:

- Локальный
- Удаленный
- Высокоуровневый

Межпроцессное взаимодействие

Средства локального уровня IPC привязаны к процессору и возможны только в пределах компьютера

К этому виду IPC принадлежат практически все основные механизмы IPC, а именно, каналы, разделяемая память и очереди сообщений

Коммуникационное пространство этих IPC поддерживаются только в пределах локальной системы. Из-за этих ограничений для них могут реализовываться более простые и более быстрые интерфейсы

Межпроцессное взаимодействие

Удаленные IPC предоставляют механизмы, которые обеспечивают взаимодействие как в пределах одного процессора, так и между программами на различных процессорах, соединенных через сеть

Сюда относятся удаленные вызовы процедур (Remote Procedure Calls - RPC), сокеты Unix, а также TLI (Transport Layer Interface - интерфейс транспортного уровня) фирмы Sun

Под высокоуровневыми IPC обычно подразумеваются пакеты программного обеспечения, которые реализуют промежуточный слой между системной платформой и приложением

Межпроцессное взаимодействие

Какие вообще бывают IPC механизмы?

Все IPC механизмы можно разделить на следующие категории:

- Взаимодействие
- Синхронизация
- Сигналы

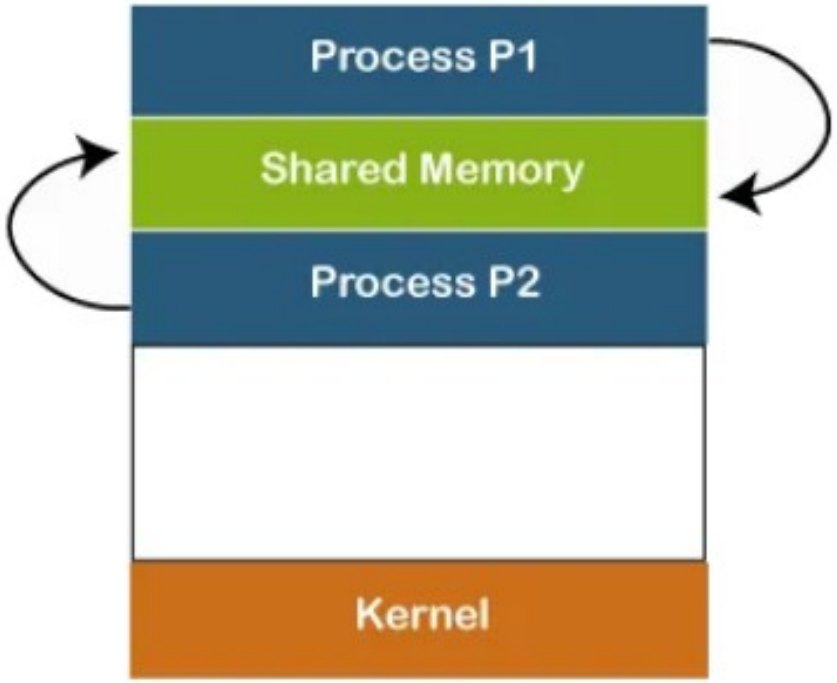
Межпроцессное взаимодействие

Средства взаимодействия можно разделить на две категории:

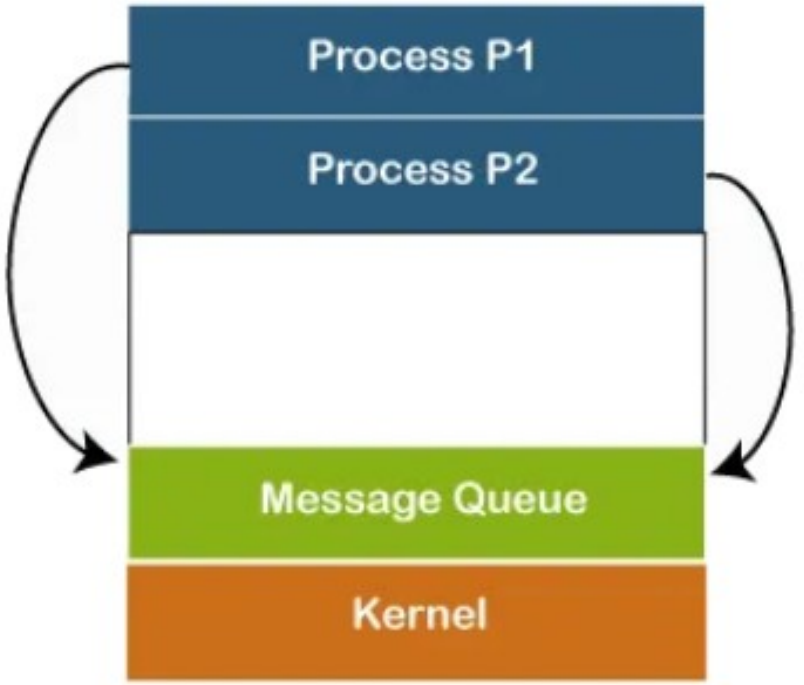
- Средства передачи данных
  - Байтовый поток
  - Сообщение
  - Псевдотерминалы
- Разделяемая память

Взаимодействие

Глобальные подходы к взаимодействию между процессами

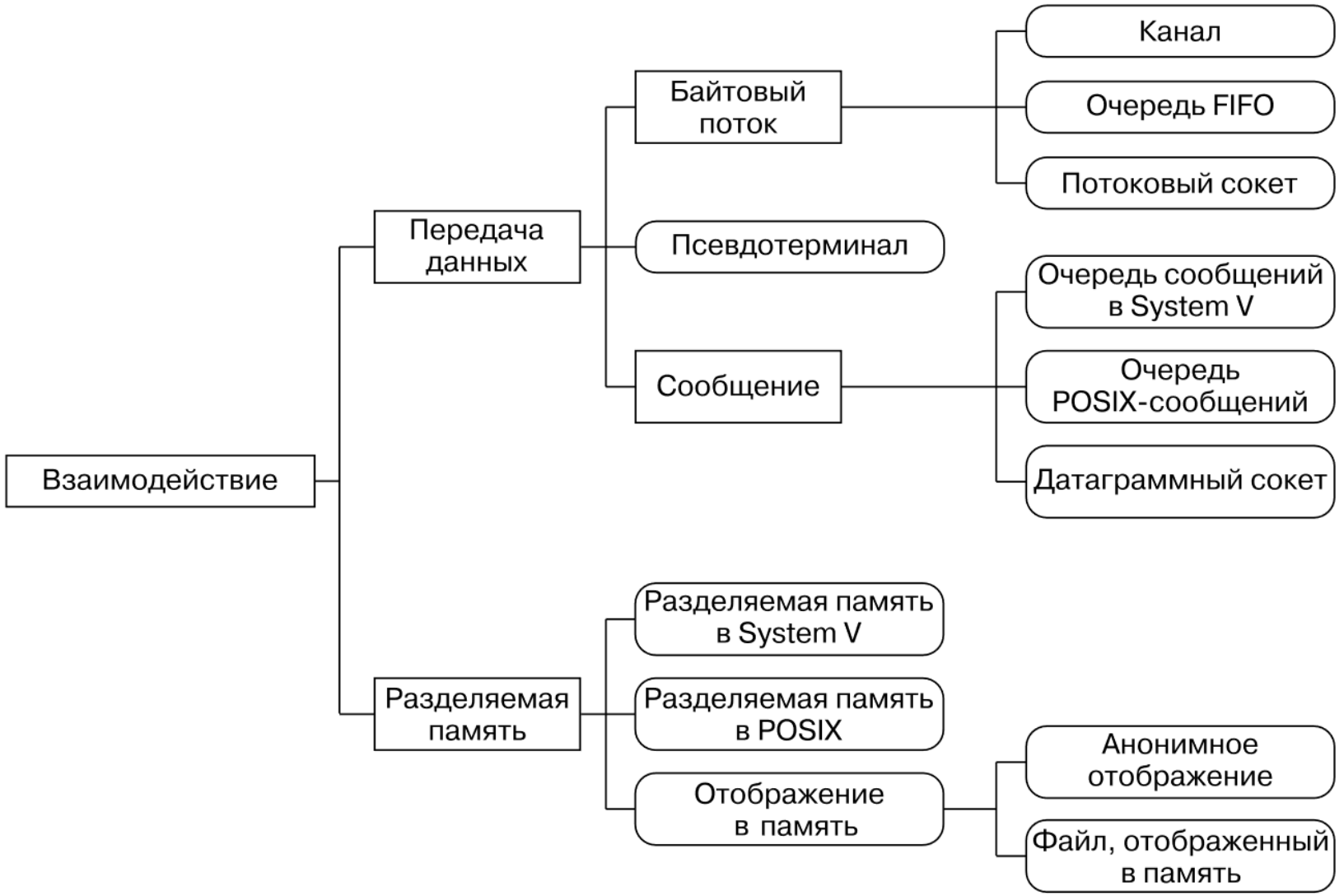


Разделяемая память



Передача данных

Взаимодействие



Взаимодействие – Что выбрать?

Передача данных	Разделяемая память
Процессы должны быть слабосвязанными	Требования к повышенной производительности
Процессы являются частью децентрализованной системы	Обмен большими данными
Требования к масштабированию	Требуется гибкость при работе с данными
Требования к повышенной устойчивости к сбоям	Частая сериализация/десериализация данных
Упрощенная синхронизация	

Передача данных

Общие концепции:

Процесс, который посылает данные другому потоку, называется **отправителем**

Процесс, который получает данные от другого потока, называется **адресатом** или **получателем**

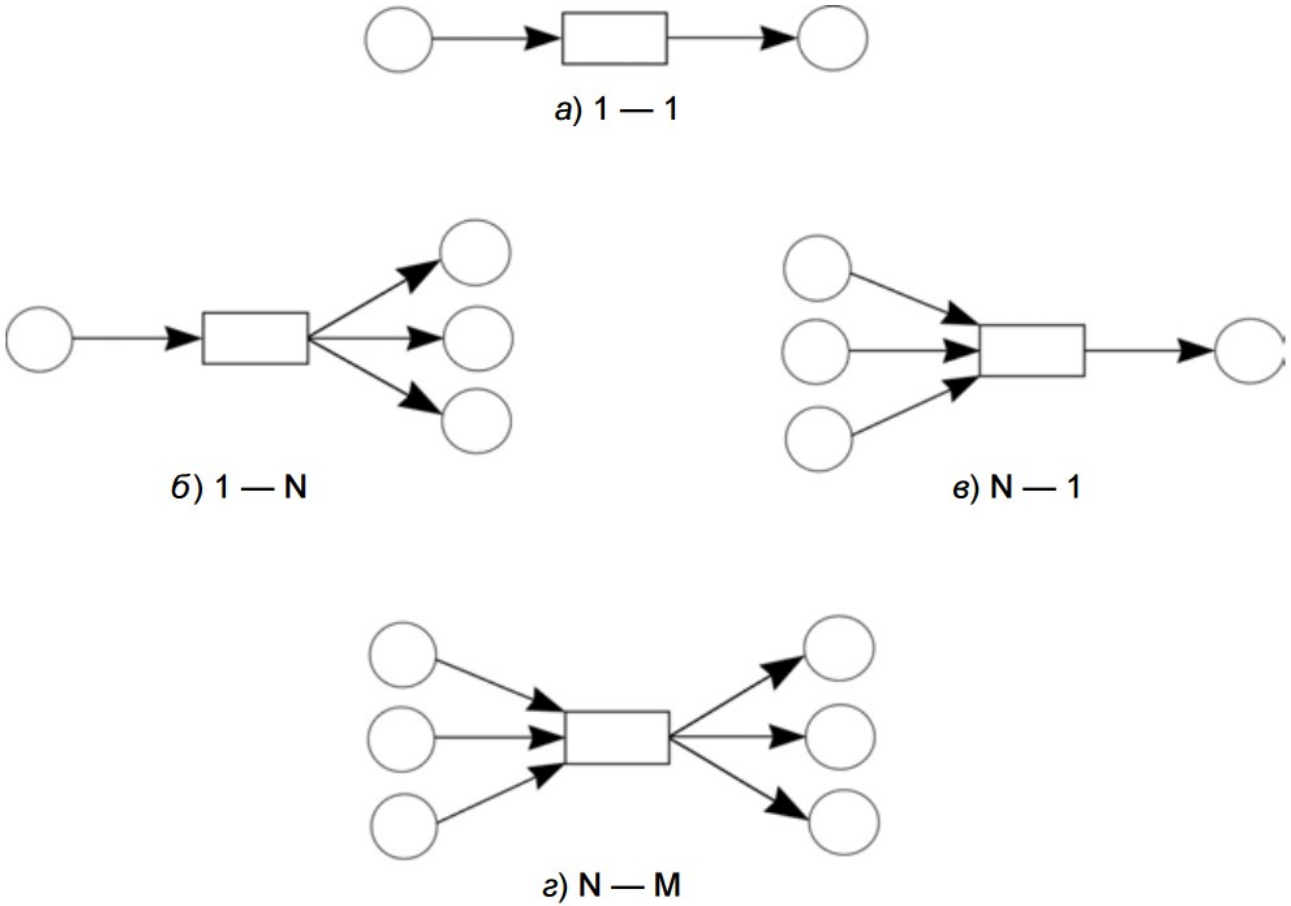
С точки зрения направления передачи данных различают следующие виды связей:

- **Симплексная** – передача только, в одном направлении
- **Полудуплексная** – передача в обе стороны, но одновременно только в одну сторону
- **Дуплексная** – передача в двух направлениях одновременно

Передача данных

Под **топологией** связи будем понимать конфигурацию связей между процессами-отправителями и адресатами.

С точки зрения топологии различают следующие виды связей:



Передача данных

Концептуально обмен данными между процессами выполняется при помощи двух функций:

send – послать данные

receive – получить данные

При передаче данных может использоваться **прямая** или **косвенная** адресация процессов

При прямой адресации процессов в функциях send и receive явно указываются процессы отправитель и адресат

При косвенной адресации в функциях send и receive указываются не адреса, а имя связи, по которой передаются данные

Передача данных

Адресация процессов может быть **симметричной** и **асимметричной**. Если при обмене данными между процессами используется только прямая или только косвенная адресация, то такая адресация процессов называется симметричной. Если же при обмене данными между процессами используется как прямая, так и косвенная адресация, то такая адресация процессов называется асимметричной

Например, ассиметричной является связь «клиент-сервер»

Передача данных

При передаче данных различают **синхронный** и **асинхронный** обмен данными

Синхронная отправка – отправитель при отправке данных, блокируется до получения этих данных адресатом

Синхронное получение – адресат вызывая ***receive*** блокируется до тех пор, пока не получит данные

Асинхронные варианты данных операций не приводят к блокировке отправителя и адресата соответственно

(А)синхронный обмен = (А)синхронная отправка + (А)синхронное получение

В целом в системах может использоваться **смешанный** обмен данными

Передача данных

Буфером называется вместимость связи между процессами, т. е. количество данных, которые могут одновременно пересылаться по этой связи. Различаются три типа буферизации:

- **Нулевая вместимость связи** (нет буфера) – в этом случае возможен только синхронный обмен данными между процессами
- **Ограниченная вместимость связи** (ограниченный буфер) – в этом случае, если буфер полон, то отправитель данных должен ждать очистки буфера хотя бы от части данных
- **Неограниченная вместимость связи** (неограниченный буфер) – в этом случае отправитель никогда не ждет при отправке сообщения

Передача данных – Канал

Канал (pipe) – это однонаправленный канал связи, который позволяет передавать данные между двумя связанными процессами

Свойства присущие любым каналам:

- Полудуплексные
- Передача данных потоком
- Синхронный обмен данными
- Возможность моделирования любой топологии связей

Передача данных – Канал

Каналы бывают **анонимные** и **именованные**

Анонимные каналы применяются только в рамках родительски-дочерних отношений между процессами, откуда следует, что данные каналы могут применяться только в рамках одного устройства

Именованные каналы могут применяться между независимыми процессами, а также могут использоваться между процессами на разных устройствах (по сети)

Передача данных – Канал

Именованные каналы обладают дополнительными свойствами:

- Могут быть дуплексными
- Передача данных может быть и поточная, и сообщениями
- Есть возможность асинхронного обмена данными
- Имеют имя формата: «\\.\pipe\pipe_name» **(<u>Windows</u>)**, в Linux имя не имеет строгого формата

Передача данных – Канал (Windows)

Алгоритмы работы с каналами

Анонимный	Именованный
создание анонимного канала сервером	создание именованного канала сервером
соединение клиентов с каналом	соединение сервера с экземпляром именованного канала
обмен данными по каналу	соединение клиента с экземпляром именованного канала
закрытие канала	обмен данными по именованному каналу
	отсоединение сервера от экземпляра именованного канала
	закрытие именованного канала клиентом и сервером

Передача данных – Канал (Windows)

Алгоритмы работы с каналами (**Подробнее о работе с каналами**)

Анонимный	Именованный
CreatePipe	**CreateNamedPipeW**
Передача дескрипторов через консоль или STARTUPINFO	**ConnectNamedPipe**
ReadFile + WriteFile	**WaitNamedPipe + CreateFile**
CloseHandle	**ReadFile + WriteFile + PeekNamedPipe**
	DisconnectNamedPipe
	CloseHandle

Передача данных – Канал (POSIX)

Алгоритмы работы с каналами:

- Создание канала
- В случае с именованными – открытие канала на чтение или запись
- Обмен данными
- Закрытие канала

Передача данных – Канал (POSIX)

Алгоритмы работы с каналами

Анонимный	Именованный
pipe	**mkfifo**
Дескрипторы доступны дочерним по умолчанию	**open**
read + write	**read + write**
close	**close**

Передача данных – Почтовый ящик

Почтовый ящик (**Mailslot**) – объект ядра операционной системы Windows, который обеспечивает передачу сообщений от процессов-клиентов к процессам-серверам, выполняющимся на компьютерах в пределах локальной сети

Почтовые ящики обладают следующими свойствами:

- Передача данных осуществляется сообщениями
- Направление передачи данных от клиента к серверу
- обмен данными может быть как синхронным, так и асинхронным
- Имеют имя формата: «\\.\mailslot\mailslot_name»
- При размере сообщения до 425 Байт отправка широковещательная
- При размере сообщения от 426 Байт до 64 Кбайт – отправка от одного клиента одному серверу

Передача данных – Почтовый ящик

Алгоритм работы с почтовыми ящиками:

- Создание почтового ящика сервером – **CreateMailslot**
- Соединение клиента с почтовым ящиком – **CreateFile**
- Обмен данными через почтовый ящик – **ReadFile** + **WriteFile**
- Закрытие почтового ящика клиентом и сервером – **CloseHandle**

Передача данных – Очередь сообщений

Очередь сообщений – IPC механизм использующийся для передачи информации между процессами. Считывающий и записывающий процессы обмениваются блоками (сообщениями) с четкими границами (в отличие от каналов, которые предоставляют сплошной байтовый поток)

Стандарт POSIX позволяет назначать каждому сообщению отдельный приоритет; сообщения с более высоким приоритетом передаются раньше остальных

Передача данных – Очередь сообщений

Алгоритм работы с очередью сообщений:

- Создание почтового ящика сервером – **mq_open**
- Соединение клиента с почтовым ящиком – **mq_open**
- Обмен данными через почтовый ящик – **mq_send** + **mq_receive**
- Закрытие почтового ящика клиентом и сервером – **mq_close** + **mq_unlink**
- Оповещение о появлении сообщения – **mq_notify**

Разделяемая память

Разделяемая память – это тип механизма IPC, который позволяет нескольким процессам делиться общей областью памяти, обеспечивая быструю и эффективную передачу данных между процессами

Свойства разделяемой памяти:

- Дуплексный обмен данными
- Синхронный обмен данными
- Возможность моделирования любой топологии связей

Разделяемая память

Алгоритм работы с разделяемой памятью:

- Создание объекта разделяемой памяти
- Соединение с областью памяти
- Обмен данными
- Закрытие объекта общей памяти

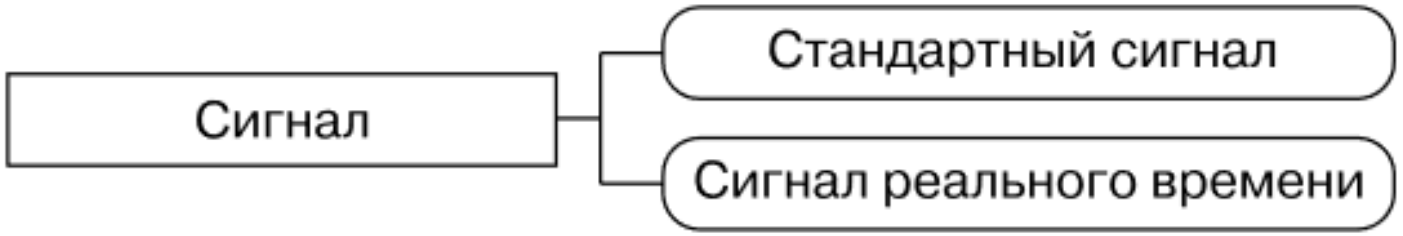
Разделяемая память

Алгоритм работы с общей памятью

Windows	Linux
CreateFileMapping + OpenFileMapping	**shm_open**
MapViewOfFile	**mmap**
Работа с адресами на разделяемом участке памяти	
UnmapViewOfFile	**munmap**
CloseHandle	**close + shm_unlink**

Сигнал

Сигнал – это оповещение процесса о том, что произошло некое событие. Иногда сигналы также описываются как программные прерывания. Сигналы аналогичны аппаратным прерываниям в том смысле, что они останавливают нормальное выполнение программы. В большинстве случаев невозможно предсказать, когда именно будет доставлен тот или иной сигнал



Сигнал

Когда сигнал послан процессу, операционная система прерывает выполнение процесса, при этом, если процесс установил собственный обработчик сигнала, операционная система запускает этот обработчик, передав ему информацию о сигнале, если процесс не установил обработчик, то выполняется обработчик по умолчанию

Сигналы могут возникать синхронно с ошибкой в приложении, например SIGFPE (ошибка вычислений с плавающей запятой) и SIGSEGV (ошибка адресации), но большинство сигналов является асинхронными

Сигнал

Сигналы могут посылаться процессу, если система обнаруживает программное событие, например, когда пользователь дает команду прервать или остановить выполнение, или получен сигнал на завершение от другого процесса

Сигналы могут прийти непосредственно от ядра ОС, когда возникает сбой аппаратных средств ЭВМ

Система определяет набор сигналов, которые могут быть отправлены процессу

Сигнал

Отдельные сигналы подразделяются на три класса:

- Системные сигналы (ошибка аппаратуры, системная ошибка и т.д.)
- Сигналы от устройств
- Сигналы, определенные пользователем

Механизм передачи сигналов состоит из следующих частей:

- Установление и обозначение сигналов в форме целочисленных значений
- Маркер в строке таблицы процессов для прибывших сигналов
- Таблица с адресами функций, которые определяют реакцию на прибывающие сигналы

Сигнал

Как только сигнал приходит, он отмечается записью в таблице процессов. Если этот сигнал предназначен для процесса, то по таблице указателей функций в структуре описания процесса выясняется, как нужно реагировать на этот сигнал. При этом номер сигнала служит индексом таблицы

Известно три варианта реакции на сигналы:

- Вызов собственной функции обработки
- Игнорирование сигнала (не работает для SIGKILL)
- Использование предварительно установленной функции обработки по умолчанию

Сигнал

Сигналы посылаются:

- С терминала, нажатием специальных клавиш или комбинаций (например, нажатие Ctrl-C генерирует SIGINT, Ctrl-\ SIGQUIT, а Ctrl-Z SIGTSTP)
- Ядром системы:
  - при возникновении аппаратных исключений (недопустимых инструкций, нарушениях при обращении в память, системных сбоях и т. п.)
  - ошибочных системных вызовах
  - для информирования о событиях ввода-вывода
- Одним процессом другому (или самому себе):
  - из C-кода с помощью системного вызова kill()
  - из shell, утилитой /bin/kill (она позволяет задавать сигнал как числом, так и символьным обозначением)

Реализация процессов

Для реализации модели процессов операционная система ведет таблицу (состоящую из массива структур), называемую таблицей процессов, в которой каждая запись соответствует какому-нибудь процессу. (Ряд авторов называют эти записи блоками управления процессом.) Эти записи содержат важную информацию о состоянии процесса ключая счетчик команд, указатель стека, распределение памяти, состояние открытых им файлов, его учетную и планировочную информацию и все остальное, касающееся процесса, что должно быть сохранено, когда процесс переключается из состояния выполнения в состояние готовности или блокировки, чтобы позже он мог возобновить выполнение, как будто никогда не останавливался.

Управление процессом	Управление памятью	Управление файлами
Регистры	Указатель на информацию о текстовом сегменте	Корневой каталог
Счетчик команд	Указатель на информацию о сегменте данных	Рабочий каталог
Слово состояния программы	Указатель на информацию о сегменте стека	Дескрипторы файлов
Указатель стека		Идентификатор пользователя
Состояние процесса		Идентификатор группы
Приоритет		
Параметры планирования		
Идентификатор процесса		
Родительский процесс		
Группа процесса		
Сигналы		
Время запуска процесса		
Использованное время процессора		
Время процессора, использованное дочерними процессами		
Время следующего аварийного сигнала		

1	Оборудование помещает в стек счетчик команд и т. п.
2	Оборудование загружает новый счетчик команд из вектора прерывания
3	Процедура на ассемблере сохраняет регистры
4	Процедура на ассемблере устанавливает указатель на новый стек
5	Запускается процедура на языке С, обслуживающая прерывание (как правило, она считывает входные данные и помещает их в буфер)
6	Планировщик принимает решение, какой процесс запускать следующим
7	Процедура на языке С возвращает управление ассемблерному коду
8	Процедура на ассемблере запускает новый текущий процесс

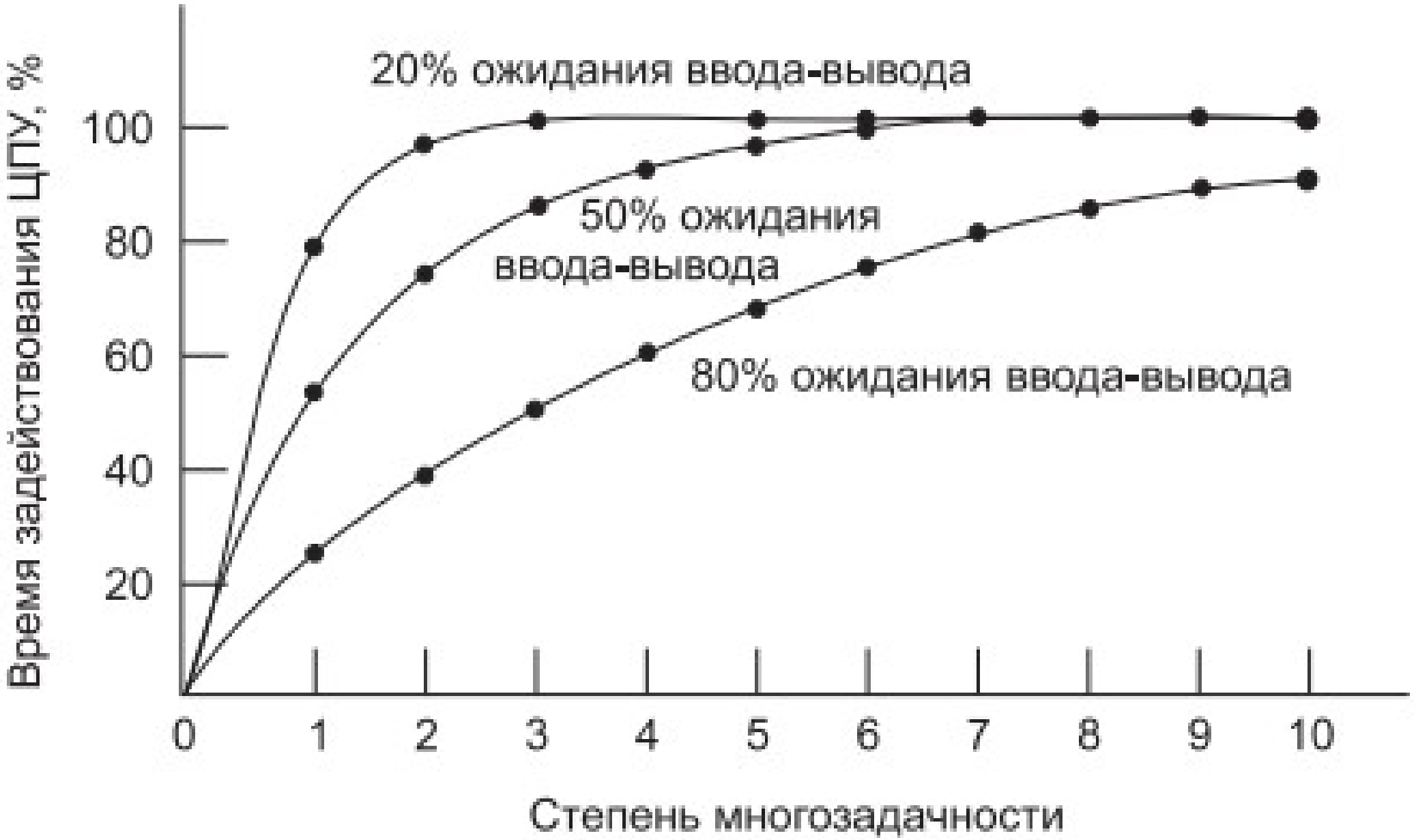
Моделирование режима многозадачности

Режим многозадачности позволяет использовать центральный процессор более рационально. При грубой прикидке, если для среднестатистического процесса вычисления занимают лишь 20 % времени его пребывания в памяти, то при пяти одновременно находящихся в памяти процессах центральный процессор будет загружен постоянно.

Время задействования ценрального процессора = 1 − p^n .

Моделирование режима многозадачности

Если процесс тратит 80 % своего времени на ожидание завершения ввода-вывода, то для снижения простоя процессора до уровня не более 10 % в памяти могут одновременно находиться по крайней мере 10 процессов. Когда вы поймете, что к ожиданию ввода-вывода относится и ожидание интерактивного процесса пользовательского ввода с терминала (или щелчка кнопкой мыши на значке), станет понятно, что время ожидания завершения ввода-вывода, составляющее 80 % и более, не такая уж редкость. Но даже на серверах процессы, осуществляющие множество операций ввода-вывода, зачастую имеют такой же или даже больший процент простоя.


20% ожидания ввода-вывода
50% ожидания ввода-вывода
80% ожидания ввода-вывода
Время задействования ЦПУ, %
100
80
60
40
20
0
1
2
3
4
5
6
7
8
9
10
Степень многозадачности

Операционные системы

Лекция 4

Управление процессами